

LẬP TRÌNH CHO TRÍ TUỆ NHÂN TẠO

Giảng viên: Nguyễn Ngọc Giang

Ôn tập lại ngôn ngữ lập trình Python

Giới thiệu ngôn ngữ python

- Python lần đầu được giới thiệu vào tháng 12/1989
- Tác giả là Guido van Rossum (Hà Lan)
 - Sinh năm 1956
 - Hiện đang làm cho Google
- Python kế thừa từ ngôn ngữ ABC
- Python 2 được giới thiệu năm 2000
 - Hỗ trợ unicode
 - Mã python 2 rất phổ biến
- Python 3 được phát hành năm 2008
 - Hiện đã có phiên bản 3.13
- Python 4? Năm 2026 (dự kiến)



Giới thiệu ngôn ngữ python

- Là ngôn ngữ có mã nguồn mở
- Là ngôn ngữ kịch bản (scripting programming language)
 - Thích hợp với DevOps (người viết code cũng là người vận hành)
 - Khai báo biến tự nhiên, phong phú và động
 - Nhiều phép tính cấp cao được cung cấp sẵn
 - Thường được thông dịch thay vì biên dịch
 - Biên dịch: dịch toàn bộ thành mã máy rồi thực thi
 - Thông dịch: dịch từng lệnh, xong lệnh nào chạy lệnh đó
- Những người cuồng python (pythonista) cho rằng ngôn ngữ này trong sáng và tiện dụng đến mức ta có thể dùng nó cho mọi khâu lập trình (chứ không phải chỉ viết script)

Giới thiệu ngôn ngữ python

- Vừa hướng thủ tục, vừa hướng đối tượng
- Hỗ trợ module và hỗ trợ gói (package)
- Xử lý lỗi bằng ngoại lệ (exception)
- Kiểu dữ liệu động ở mức cao
- Có khả năng tương tác với các module viết bằng ngôn ngữ lập trình khác
- Có thể nhúng vào ứng dụng như một giao tiếp kịch bản (scripting interface)

Ưu điểm của ngôn ngữ python

- Có ngữ pháp đơn giản, dễ đọc
- Viết mã ngắn gọn hơn những chương trình tương đương được viết trong C, C++, C#, Java,...
- Có các bộ thư viện chuẩn và các module ngoài, đáp ứng gần như mọi nhu cầu lập trình
- Có khả năng chạy trên nhiều nền tảng (Windows, Linux, Unix, OS/2, Mac, Amiga, máy ảo .NET, máy ảo Java, Nokia Series 60,...)
- Có cộng đồng lập trình rất lớn, hệ thống thư viện chuẩn, mã nguồn chia sẻ nhiều

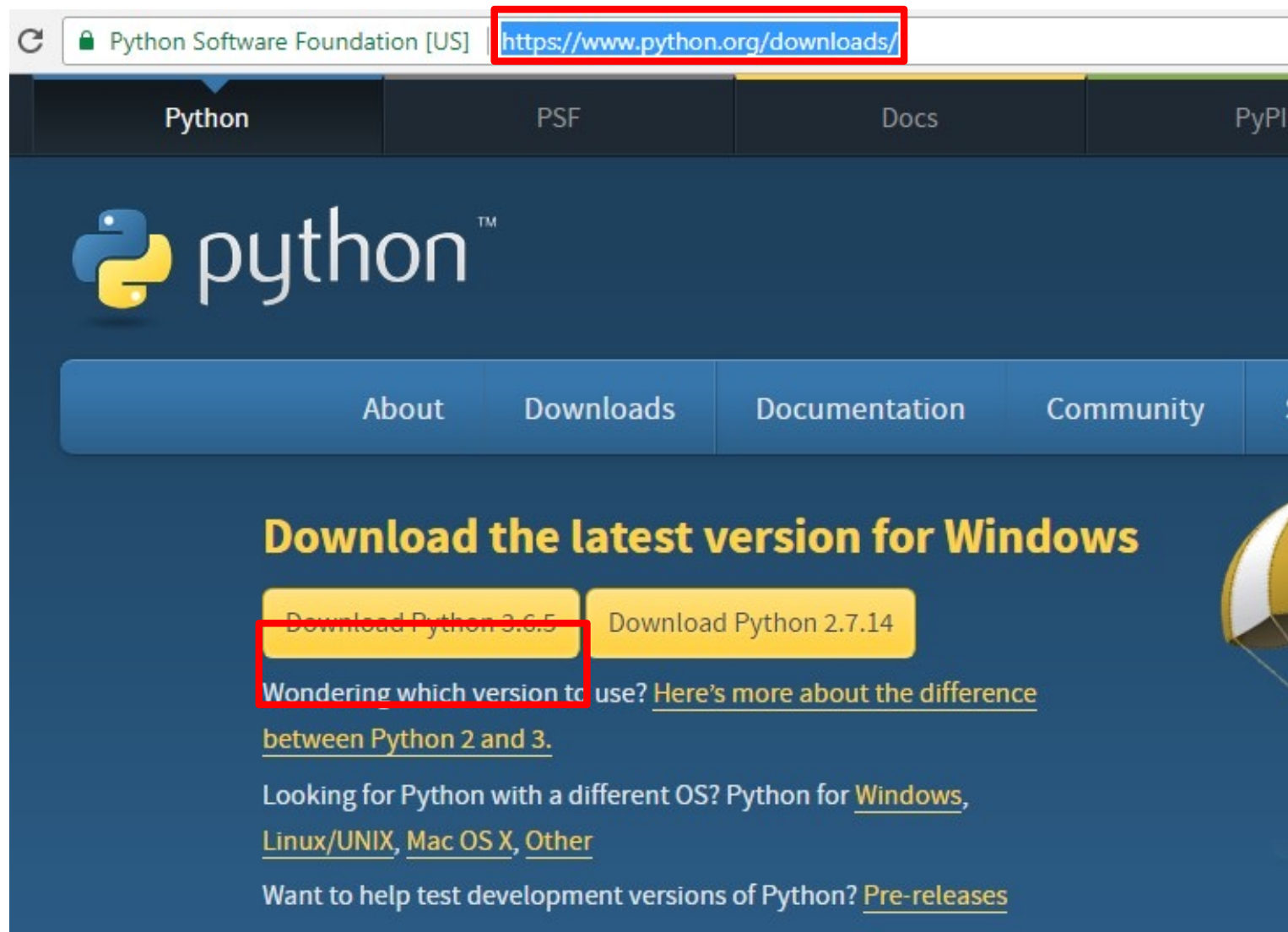
Nhưng python cũng có nhược điểm

- Chương trình chạy chậm
- Yếu trong hỗ trợ tính toán trên di động
- Gỡ lỗi đòi hỏi kinh nghiệm

Phần 1

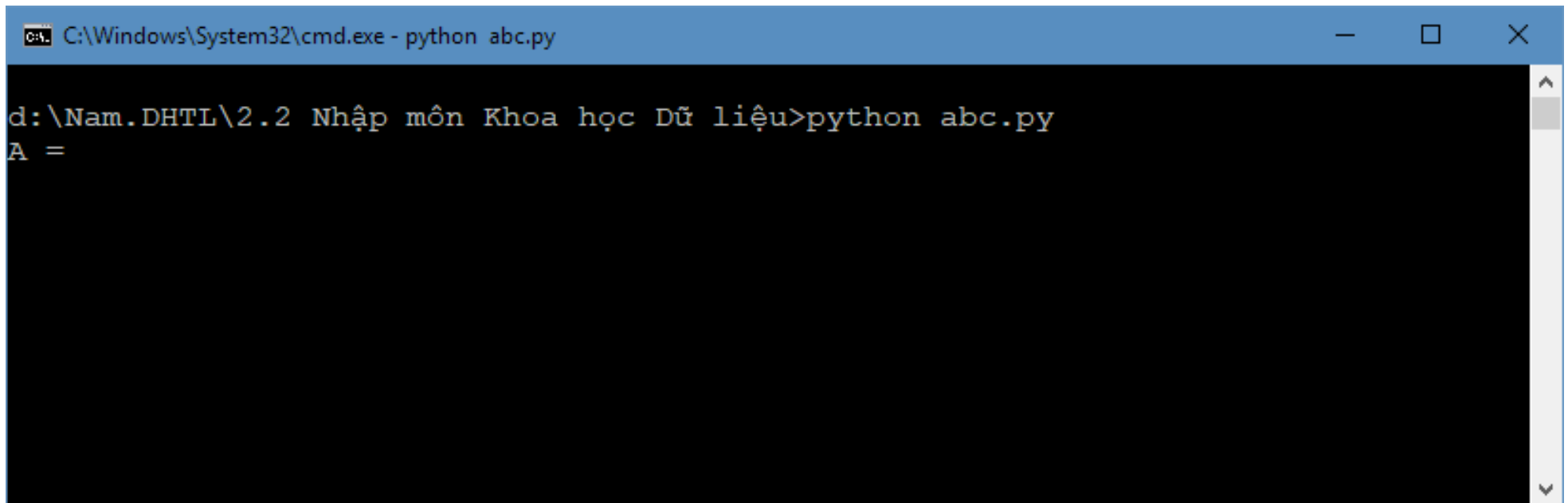
Cách thực hiện câu lệnh, chương trình

Cài đặt



Khởi chạy

- Python có 2 chế độ thực thi
 - Chế độ chương trình: chỉ ra chương trình cần thực hiện
 - Trình dịch python sẽ nạp, dịch và chạy chương trình đó
 - Chế độ dòng lệnh: gõ và chạy từng lệnh một
- Chế độ thực thi: “python abc.py” chạy file abc.py

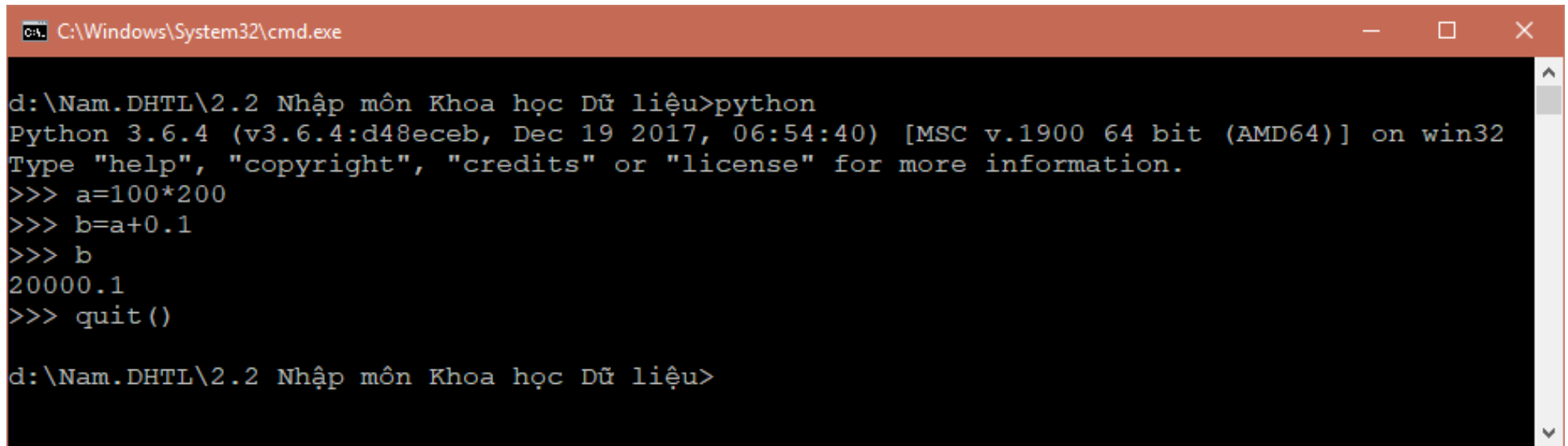


```
C:\Windows\System32\cmd.exe - python abc.py

d:\Nam.DHTL\2.2 Nhập môn Khoa học Dữ liệu>python abc.py
A =
```

Khởi chạy

- Chế độ dòng lệnh: “python”
 - Lúc này trình thông dịch python sẽ chờ người dùng gõ từng dòng lệnh
 - Gõ dòng lệnh nào xong, python chạy liền dòng đó
 - Chấm dứt chế độ này bằng cách gõ lệnh: “quit()”



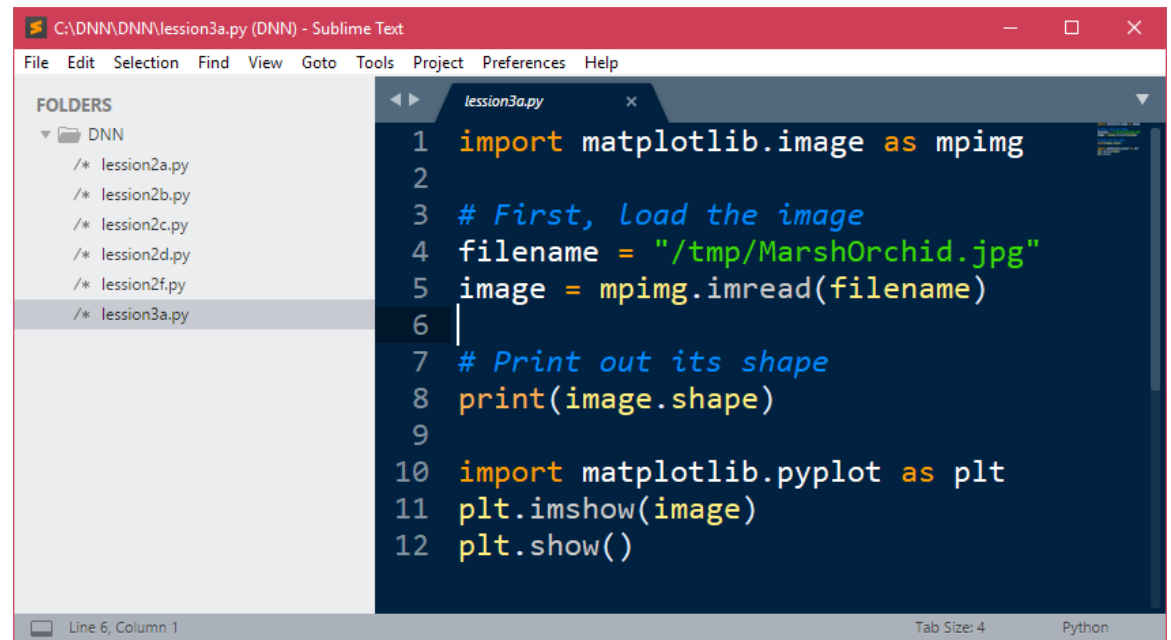
```
C:\Windows\System32\cmd.exe

d:\Nam.DHTL\2.2 Nhập môn Khoa học Dữ liệu>python
Python 3.6.4 (v3.6.4:d48eceb, Dec 19 2017, 06:54:40) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> a=100*200
>>> b=a+0.1
>>> b
20000.1
>>> quit()

d:\Nam.DHTL\2.2 Nhập môn Khoa học Dữ liệu>
```

Soạn thảo mã python

- Làm thế nào để viết chương trình python (.py)?
 - Dùng phần mềm soạn thảo văn bản thô (txt) bất kỳ để soạn và lưu file ở dạng .py rồi dịch bằng python
- Có những phần mềm thích hợp cho việc này hơn
 - IDLE
 - Sublime Text
 - Notepad++
 - PyCharm
 - Spyder
 - Rodeo
 - ...
- Jupyter: chạy trên trình duyệt, thích hợp với thử nghiệm và giảng dạy



The screenshot shows the Sublime Text editor interface. The title bar indicates the file path is 'C:\DNN\DNN\lesson3a.py (DNN) - Sublime Text'. The menu bar includes File, Edit, Selection, Find, View, Goto, Tools, Project, Preferences, and Help. On the left, a 'FOLDERS' panel shows a tree view with 'DNN' expanded, listing files: lesson2a.py, lesson2b.py, lesson2c.py, lesson2d.py, lesson2f.py, and lesson3a.py. The main editor area displays the content of 'lesson3a.py' with the following code:

```
1 import matplotlib.image as mpimg
2
3 # First, load the image
4 filename = "/tmp/MarshOrchid.jpg"
5 image = mpimg.imread(filename)
6
7 # Print out its shape
8 print(image.shape)
9
10 import matplotlib.pyplot as plt
11 plt.imshow(image)
12 plt.show()
```

The status bar at the bottom shows 'Line 6, Column 1', 'Tab Size: 4', and 'Python'.

Biên dịch mã python

- Trường hợp cần thiết, mã python có thể được biên dịch, kết quả biên dịch là chương trình dạng bytecode cho máy ảo python
 - Tương tự như trường hợp của ngôn ngữ java
- Mã lệnh dịch được lưu vào file với đuôi `.pyc`
- Việc biên dịch có nhiều lợi điểm, chẳng hạn như khi sử dụng câu lệnh `import` một thư viện nào đó, thì có thể sử dụng luôn mã pyc có sẵn thay vì phải dịch lại từ đầu
 - Tăng tốc độ thực hiện chương trình

Phần 2

Biến, khai báo chuỗi, khối lệnh

Biến

- Biến = vùng bộ nhớ được đặt tên (để dễ thao tác)

- Ví dụ:

```
n = 12          # biến n là kiểu nguyên  
n = n + 0.1     # biến n chuyển sang kiểu thực
```

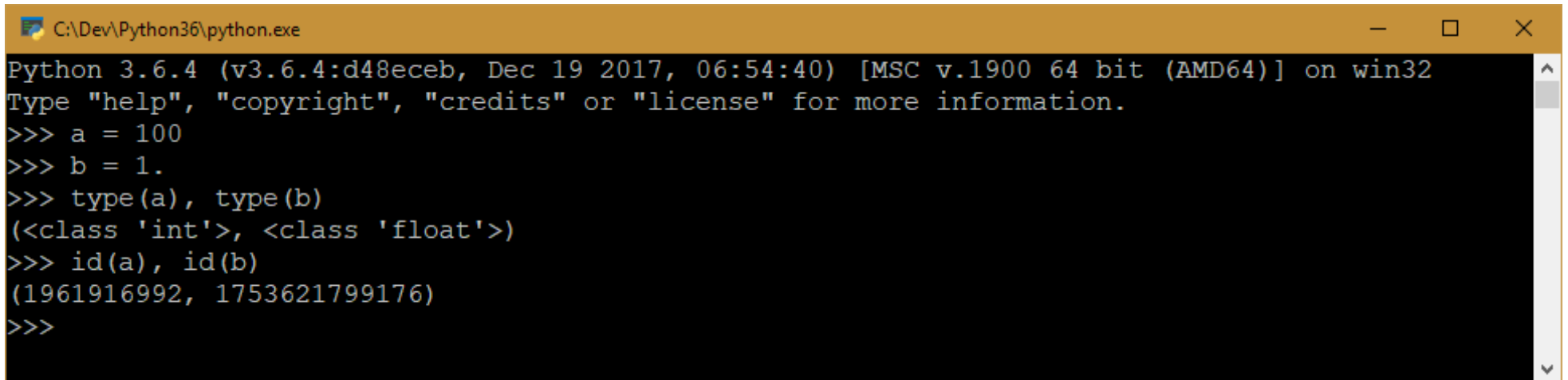
- Biến trong python:

- Có tên, phân biệt chữ hoa/thường
- Không cần khai báo trước
- Không cần chỉ ra kiểu dữ liệu
- Có thể thay đổi sang kiểu dữ liệu khác
- Nên gán giá trị ngay khi bắt đầu xuất hiện

- Chú ý: python cho phép viết ghi chú trong chương trình bằng cách đặt sau dấu thăng (#)

Biến

- Tên biến có thể chứa chữ cái hoặc chữ số hoặc gạch dưới (`_`), kí tự bắt đầu không được dùng chữ số
 - Không được trùng với từ khóa (tất nhiên)
 - Từ python 3 được dùng chữ cái unicode
- Tất cả mọi biến trong python đều là các đối tượng, vì thế nó có kiểu và vị trí trong bộ nhớ (id)



```
C:\Dev\Python36\python.exe
Python 3.6.4 (v3.6.4:d48eceb, Dec 19 2017, 06:54:40) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> a = 100
>>> b = 1.
>>> type(a), type(b)
(<class 'int'>, <class 'float'>)
>>> id(a), id(b)
(1961916992, 1753621799176)
>>>
```


Khai báo chuỗi

- Dữ liệu kiểu chuỗi rất quan trọng trong lập trình python, tương tự như các ngôn ngữ lập trình khác

- Ví dụ:

```
# chuỗi thông thường
name = 'matt'
# chuỗi trong nó có chứa dấu nháy đơn
with_quote = "I ain't gonna"
# chuỗi có nội dung nằm trên nhiều dòng
longer = """This string has
multiple lines in it"""
```

- Nguyên tắc khai báo chuỗi: mở đầu sao - kết thúc vậy
 - Nội dung trên 1 dòng: dùng cặp nháy đơn (') hoặc nháy kép (")
 - Nội dung nằm trên nhiều dòng: 3 dấu nháy liên tiếp (""" / ''')

Chuỗi thoát (escape sequence)

- Escape sequence là một phương pháp để viết các kí tự đặc biệt (không thể viết theo lối thông thường)
 - Tương tự như các ngôn ngữ lập trình khác

Cách viết	Ý nghĩa	Thuật ngữ
\a	Kí tự cảnh báo (phát ra một tiếng bíp nếu in ra)	Alert
\b	Kí tự xóa trước (dịch con trỏ về phía trước 1 ô)	Backspace
\n	Kí tự dòng mới (dịch con trỏ xuống dòng dưới)	Linefeed
\r	Kí tự trở về (dịch con trỏ về đầu dòng)	Carriage return
\t	Kí tự tab (dịch con trỏ đi 1 dấu tab)	Tab
\\	Kí tự gạch chéo (\)	Backslash
\'	Kí tự dấu nháy đơn (')	Single quote
\"	Kí tự dấu nháy kép (")	Double quote
\uxxxx	Kí tự unicode bất kì có mã xxxx (dạng hex value)	

Chuỗi thô (raw string)

- Vấn đề: dễ nhầm lẫn khi các chuỗi có dấu gạch chéo (\)
 - Chẳng hạn như khi viết tên file "c:\teamview"
- Python cho phép bỏ qua các chuỗi thoát bằng cách đánh dấu chữ r vào trước chuỗi, định dạng này gọi là chuỗi thô
 - Cú pháp: r' nội dung chuỗi '

```
>>> a = 'c:\teamview'
>>> print(a)
c:      eamview
>>> b = 'c:\\teamview'
>>> print(b)
c:\teamview
>>> c = r'c:\teamview'
>>> print(c)
c:\teamview
```

Khối lệnh

- Python sử dụng khoảng trắng để phân biệt khối lệnh

```
age = int(input("Bạn bao nhiêu tuổi? "))
print("Ồ bạn đã", age, "tuổi rồi!")
if age >= 18:
    print("Đủ tuổi đi bầu")
    if age > 100:
        print("Có vẻ sai sai!")
else:
    print("Nhỏ quá")
```

- Chú ý:
 - Không quy định số lượng khoảng trắng phải sử dụng
 - Các lệnh cùng một khối phải sử dụng cùng số khoảng trắng
 - Sử dụng tab hoặc space đều được, nhưng phải thống nhất

Phần 3

Nhập dữ liệu và xuất dữ liệu

Xuất dữ liệu

- Sử dụng hàm print để in dữ liệu ra màn hình

```
>>> print(42)
```

```
42
```

```
>>> print("a = ", a)
```

```
a = 3.564
```

```
>>> print("a = \n", a)
```

```
a =
```

```
3.564
```

```
>>> print("a", "b")
```

```
a b
```

```
>>> print("a", "b", sep="")
```

```
ab
```

```
>>> print(192, 168, 178, 42, sep=".")
```

```
192.168.178.42
```

```
>>> print("a", "b", sep=":-)")
```

```
a:-)b
```

Nhập dữ liệu

- Sử dụng hàm input để nhập dữ liệu từ bàn phím

```
name = input("Tên bạn là gì? ")  
print("Xin chào bạn " + name + " !")  
age = input("Bạn bao nhiêu tuổi? ")  
print("Ồ, bạn đã " + age + " tuổi rồi!")
```

- Có thể kết hợp chuyển kiểu nếu muốn tương minh

```
age = int(input("Bạn bao nhiêu tuổi? "))  
print("Ồ bạn đã %d tuổi rồi!" % age)
```

Phần 4

Kiểu dữ liệu và phép toán liên quan

Kiểu số

- Python cho phép viết số nguyên theo một số hệ cơ số thông dụng trong lập trình

`A = 1234` # hệ cơ số 10

`B = 0xAF1` # hệ cơ số 16

`C = 0o772` # hệ cơ số 8

`D = 0b1001` # hệ cơ số 2

- Sử dụng các hàm phù hợp để chuyển đổi từ số nguyên thành string ở các hệ cơ số 10, 16, 8 hoặc 2

`K = str(1234)` # chuyển thành str ở hệ cơ số 10

`L = hex(1234)` # chuyển thành str ở hệ cơ số 16

`M = oct(1234)` # chuyển thành str ở hệ cơ số 8

`N = bin(1234)` # chuyển thành str ở hệ cơ số 2

Kiểu số

- Từ python 3, số nguyên không có giới hạn số chữ số
- Số thực (float) trong python có thể viết theo dạng thông thường hoặc dạng khoa học

```
X = 12.34
```

```
Y = 314.15279e-2 # dạng số nguyên và phần mũ 10
```

- Python hỗ trợ kiểu phức, với chữ j đại diện cho phần ảo

```
A = 3+4j
```

```
B = 2-2j
```

```
print(A+B) # sẽ in ra (5+2j)
```

Phép toán

- Python hỗ trợ nhiều phép toán số, logic, so sánh, phép toán bit và phép kiểm tra tập
 - Các phép toán số thông thường: `+`, `-`, `*`, `%`, `**`
 - Python có 2 phép chia:
 - Chia đúng (`/`): `10/3` `# 3.3333333333333335`
 - Chia nguyên (`//`): `10//3` `# 3` (nhanh hơn phép `/`)
 - Các phép logic: `and`, `or`, `not`
 - Python không có phép `xor` logic, trường hợp muốn tính phép xor thì thay bằng phép so sánh khác (`bool(a) != bool(b)`)
 - Các phép so sánh: `<`, `<=`, `>`, `>=`, `!=`, `==`
 - Các phép toán bit: `&`, `|`, `^`, `~`, `<<`, `>>`
 - Phép kiểm tra tập (`in`, `not in`): `1 in [1, 2, 3]`

Phần 5

Vài ví dụ minh họa

Giải phương trình bậc 2

```
a = float(input("A = "))  
b = float(input("B = "))  
c = float(input("C = "))  
delta = b*b-4*a*c
```

Nhập a,b,c kiểu số thực và tính delta

```
if delta==0:  
    print("Nghiem kep: x = ", str(-b/2/a))
```

Biện luận các trường hợp của delta

```
if delta<0:  
    print("Phuong trinh vo nghiem")
```

Các khối lệnh con được viết thụt vào so với khối cha

```
if delta>0:  
    print("X1 = " + str((-b+delta**0.5)/2/a))  
    print("X2 = " + str((-b-delta**0.5)/2/a))
```

Tính căn bậc 2 bằng phép lũy thừa 0.5

Tính $n!$

```
def giaithua(n):
```

```
    gt = 1
```

```
    for i in range(2, n+1):
```

```
        gt = gt * i
```

```
    return gt
```

```
a = int(input("Nhập giá trị n: "))
```

```
print("N! =", giaithua(a))
```

Định nghĩa hàm
với tham số n

Vòng lặp cho i
chạy từ 2 đến n

Trả về kết quả

Nhập số n nguyên

Gọi hàm tính và
in ra kết quả

Tính UCLN (thuật toán euclid)

```
a = int(input("A = "))  
b = int(input("B = "))
```

Nhập 2 số nguyên
a và b

```
while (b > 0):  
    if (a > b):  
        a, b = b, a % b  
    else:  
        a, b = a, b % a
```

Vòng lặp chừng
nào $b > 0$

Xử lý khi $a > b$

Xử lý khi $a \leq b$

```
print("Ước số chung lớn nhất là:", a)
```

In kết quả

Phần 6

Bài tập

Bài tập

1. Gõ 3 ví dụ phía trước vào 3 file, đặt tên đuôi .py, sau đó sử dụng trình thông dịch python để chạy thử xem kết quả ra sao
2. Bạn có 10 triệu đồng trong tài khoản ngân hàng, với lãi xuất 5,1% hàng năm. Tính xem:
 - Sau 10 năm bạn có bao nhiêu tiền?
 - Sau bao nhiêu năm bạn sẽ có ít nhất 50 triệu đồng?
3. Nhập số nguyên n, hãy in ra n ở dạng hệ cơ số 16, hệ cơ số 8 và hệ cơ số 2
4. Nhập 2 số nguyên a và b, hãy tính và in ra $\sqrt[b]{a}$
5. Nhập số nguyên X, hãy đếm xem X có bao nhiêu chữ số, in ra chữ số đầu tiên của X

LẬP TRÌNH PYTHON

Bài 2: Hàm và rẽ nhánh trong python

Tóm tắt nội dung bài trước

- Hai cách thực thi python: chạy chương trình và dòng lệnh
- Dùng dấu thăng (#) để viết dòng chú thích
- Biến không cần khai báo trước, không cần chỉ kiểu
- Dữ liệu chuỗi nằm trong cặp nháy đơn ('), nháy kép ("), hoặc ba dấu nháy (""" / ''') – nếu viết nhiều dòng
 - Sử dụng chuỗi thoát (\) để khai báo các ký tự đặc biệt
 - Sử dụng chuỗi thô: r"nội dung"
- Hàm print để in dữ liệu, hàm input để nhập dữ liệu
 - Có thể kết hợp với hàm chuyển đổi kiểu
- Kiểu số và phép toán có một số điểm cần chú ý
 - Số nguyên không giới hạn độ lớn
 - Phép chia nguyên và phép chia chính xác

Chữa bài tập buổi trước

Nhập 2 số nguyên a và b, hãy tính và in ra $\sqrt[b]{a}$

```
a = int(input("Nhập số nguyên A = "))  
b = int(input("Nhập số nguyên B = "))  
print("Kết quả:", a ** (1 / b))
```

Chữa bài tập buổi trước

Nhập số nguyên n, hãy in ra n ở dạng hệ cơ số 16, hệ cơ số 8 và hệ cơ số 2

```
n = int(input("Nhập số nguyên N = "))  
print("N ở hệ cơ số 16:", hex(n))  
print("N ở hệ cơ số 8:", oct(n))  
print("N ở hệ cơ số 2:", bin(n))
```

Chữa bài tập buổi trước

Bạn có 10 triệu đồng trong tài khoản ngân hàng, với lãi suất 5,1% hàng năm. Tính xem:

- Sau 10 năm bạn có bao nhiêu tiền?
- Sau bao nhiêu năm bạn sẽ có ít nhất 50 triệu đồng?

```
import math
```

```
tien = 1e7 # số tiền đầu (10M)
```

```
lai = 5.1 / 100 # lãi suất 5.1%
```

```
print("Số tiền sau 10 năm:", int(tien * (1 + lai)**10))
```

```
dich = 5e7 # số tiền đích (50M)
```

```
nam = math.log(dich / tien, 1 + lai) # tính theo log
```

```
print("Số năm để có ít nhất 50 triệu:", math.ceil(nam))
```

Chữa bài tập buổi trước

Nhập số nguyên X, hãy đếm xem X có bao nhiêu chữ số, in ra chữ số đầu tiên của X

(sinh viên chủ động giải thích cách làm dưới đây bằng kiến thức toán học cơ sở)

```
import math
```

```
x = int(input("Nhập số nguyên X = "))
```

```
len = math.floor(math.log10(x))
```

```
print("Số chữ số của X:", len + 1)
```

```
print("Chữ số đầu tiên của X:", x // 10**len)
```

Nội dung

1. Hàm
2. Phép toán “if”
3. Rẽ nhánh
4. Bài tập

Phần 1

Hàm

Khai báo và gọi hàm

- Cú pháp khai báo hàm rất đơn giản

```
def <tên-hàm>(danh-sách-tham-số):  
    <lệnh 1>  
    ...  
    <lệnh n>
```

- Ví dụ: hàm tính tích 2 số

```
def tich(a, b):  
    return a * b
```

- Hàm trả về kết quả bằng lệnh **return**, nếu không trả về thì coi như trả về **None**
- Gọi hàm thông qua tên và các đối số

```
dt = tich(100, 200)  
s = tich(20, 30) + tich(40, 50)
```

Hàm với tham số mặc định

- Hàm có thể chỉ ra giá trị mặc định của tham số

nếu không nói gì thì mặc định b = 1

```
def tinh(a, b = 1):
```

```
    return a*b
```

- Như vậy với hàm trên ta có thể gọi thực hiện nó:

```
print(tinh(10, 20))      # 200
```

```
print(tinh(10))          # 10
```

```
print(tinh(a=5))         # 5
```

```
print(tinh(b=6, a=5))    # 30
```

- Chú ý: các tham số có giá trị mặc định phải đứng cuối danh sách tham số

Trả về kết quả từ hàm

- Hàm không có kiểu, vì vậy có thể trả về bất kì loại dữ liệu gì, thậm chí có thể trả về nhiều kiểu dữ liệu khác nhau

```
def fuc1():  
    return 1001      # trả về một loại kết quả  
def fuc2():  
    print('None')    # không trả về kết quả  
def fuc3():  
    return 1001, 'abc', 4.5    # trả về phức hợp nhiều loại  
def fuc4(n):  
    if n < 0:  
        return 'số âm'        # trả về chuỗi  
    else:  
        return n + 1          # trả về số
```

Python không cho phép nạp chồng hàm

Python không cho phép hàm trùng tên, nếu cố ý định nghĩa nhiều hàm trùng tên, python sẽ sử dụng phiên bản cuối cùng

```
def abc():  
    return 'abc version 1'
```

```
def abc(a):  
    return 'abc version 2'
```

```
def abc(a, b):  
    return 'abc version 3'
```

```
print(abc())           # lỗi, hàm abc cần 2 tham số a và b
```

```
print(abc(1, 2))       # ok, in ra 'abc version 3'
```

Tham số tùy biến trong python

Python cho phép số lượng tham số tùy ý bằng cách đặt dấu sao (*) vào phía trước tên tham số.

Trong ví dụ dưới *names là một dãy không giới hạn số tham số

tham số tùy biến

```
def sayhello(*names):
```

```
    # duyệt các tham số
```

```
    for name in names:
```

```
        print("Hello", name)
```

gọi hàm với 4 tham số

```
sayhello("Monica", "Luke", "Steve", "John")
```

gọi hàm với 3 tham số

```
sayhello("Aba", "Donald", "Pence")
```

Phần 2

Câu lệnh rẽ nhánh if-elif-else

Phép toán “if”

X là max của A và B

```
X = A if A > B else B
```

N có phải là số nguyên tố có 1 chữ số hay không

```
A = "Đúng" if N in [2, 3, 5, 7] else "Sai"
```

In ra màn hình “chẵn” nếu n chia hết cho 2,

in ra “lẻ” nếu ngược lại

```
print('chẵn' if (n % 2) == 0 else 'lẻ')
```

Sinh viên có được thi hay không?

```
print("được thi" if so_buoi_nghi < 3 else "không được thi")
```

Biện luận nghiệm phương trình bậc 2 (if lồng nhau)

```
KQ = "một nghiệm" if delta == 0 else \  
      "vô nghiệm" if delta < 0 else "hai nghiệm"
```


Phép toán “if”

- Cú pháp: `A if <điều-kiện> else B`
- Thực hiện:
 - Phép toán trả về A nếu <điều-kiện> là đúng, ngược lại trả về B
 - A và B có thể là các giá trị, biểu thức tính toán, lời gọi hàm,...
 - Các phép toán if cũng có thể lồng nhau
- Cách sử dụng `if` này khá kỳ lạ, nhưng hợp lý nếu xét về mặt ngôn ngữ và cách đọc điều kiện logic
- Ưu điểm: đây là phép toán, có thể viết trong biểu thức
- Bài tập: Biến X để lưu tình trạng gửi SMS, X=0 tức là chưa gửi được, X=1 tức là đã gửi thành công, X=2 tức là đã gửi và người nhận đã đọc. Viết câu lệnh sử dụng phép toán if để in ra màn hình thông báo tương ứng với giá trị của X.

Rẽ nhánh dạng đầy đủ

```
# In thông báo nếu được điểm số loại giỏi
if diem >= 8: print("Chúc mừng bạn đã được điểm giỏi")
# In thông báo xem n chẵn hay lẻ
if (N % 2) == 0:
    print("N là số chẵn")
else:
    print("N là số lẻ")
# Biện luận nghiệm của phương trình bậc 2
if delta == 0:
    print("Phương trình có nghiệm kép")
elif delta < 0:
    print("Phương trình vô nghiệm")
else:
    print("Phương trình có hai nghiệm phân biệt")
```

Rẽ nhánh

```
if expression:  
    # If-block
```

```
if expression:  
    # If-block  
  
else:  
    # else-block
```

```
if expression:  
    # If-block  
  
elif 2-expression:  
    # 2-if-block  
  
elif 3-expression:  
    # 3-if-block  
  
...  
  
elif n-expression:  
    # n-if-block
```

```
if expression:  
    # If-block  
  
elif 2-expression:  
    # 2-if-block  
  
...  
  
elif n-expression:  
    # n-if-block  
  
else:  
    # else-block
```

Rẽ nhánh

- Python chỉ có một cấu trúc rẽ nhánh duy nhất, sử dụng để lựa chọn làm một trong số nhiều công việc
 - Nhiều ngôn ngữ lập trình khác sử dụng **if** cho trường hợp 2 lối rẽ nhánh và **switch** cho trường hợp nhiều lối rẽ nhánh
- Nguyên tắc với rẽ nhánh **if-elif-else**:
 - Biểu thức điều kiện của **if** và **elif** phải có kết quả logic
 - Hệ thống sẽ lần lượt tính giá trị từng biểu thức điều kiện từ trên xuống dưới, bắt đầu từ phát biểu **if**
 - Nếu biểu thức điều kiện nào đúng thì khối lệnh tương ứng được thực hiện và bỏ qua các khối lệnh khác
 - Trường hợp mọi biểu thức điều kiện đều sai, khối lệnh ứng với **else** sẽ được thực hiện
 - Khối **else** là tùy chọn, không nhất thiết phải xuất hiện

Phân tích ví dụ

```
a = int(input("A = "))

if 0 == a % 2:
    print("A là số chẵn")
elif 0 == a % 5:
    print("A chia hết cho 5")
else:
    print("A không có gì đặc biệt")
```

a = 11: in ra “A không có gì đặc biệt”
a = 12: in ra “A là số chẵn”
a = 15: in ra “A chia hết cho 5”
a = 10: in ra “A là số chẵn” – chú ý – A vừa là số chẵn vừa chia hết cho 5, nhưng lệnh dừng ngay khi xét điều kiện A chẵn.

Lệnh if lồng nhau, chú ý thụt lề

```
age = int(input("Bạn bao nhiêu tuổi? "))
print("Ồ bạn đã", age, "tuổi rồi!")
if age >= 18:
    print("Đủ tuổi đi bầu cử")
    if age > 100:
        print("Có vẻ sai sai!")
else:                                     # else thuộc về if ở dòng 3
    print("Nhỏ quá")
```

Như vậy trong python thì dấu cách cũng có vai trò lập trình của nó, không chỉ đơn giản là viết cho đẹp

Phần 3

Bài tập

Bài tập

1. Viết hàm `average` nhận 5 tham số và trả về giá trị trung bình cộng của chúng
2. Viết hàm `area` trả về diện tích tam giác khi biết độ dài ba cạnh của chúng
3. Viết hàm `area2` trả về diện tích tam giác biết tọa độ (x, y) của ba đỉnh tam giác
4. Viết hàm `total` nhận số nguyên N làm tham số, hàm trả về tổng các chữ số của số N (chẳng hạn N = 132 thì hàm `total` trả về 6)
5. Viết hàm `isFibo` nhận số nguyên N làm tham số, hàm kiểm tra xem N có phải số fibonacci hay không? Trả về True nếu đúng, ngược lại trả về False

Bài tập

6. Viết chương trình nhập số nguyên dương N và tính giá trị hàm $F(N)$ dưới đây

$$F(N) = 1! + 2! + \dots + N!$$

7. Viết chương trình nhập điểm trung bình học tập của một sinh viên, sau đó dựa trên quy tắc dưới đây in ra đánh giá kết quả học tập của sinh viên đó.

- Điểm dưới 3.5: xếp loại yếu
- Điểm từ 3.5 đến dưới 5: xếp loại kém
- Điểm từ 5 đến dưới 6.5: xếp loại trung bình
- Điểm từ 6.5 đến dưới 8: xếp loại khá
- Điểm từ 8 đến dưới 9: xếp loại giỏi
- Điểm từ 9 trở lên: xếp loại xuất sắc

Bài tập

8. Nhập vào từ bàn phím ba số a , b và c . Thực hiện các công việc dưới đây:

- In ra màn hình giá trị lớn nhất trong ba số
- Nếu có ít nhất hai số cùng nhận giá trị lớn nhất, in ra giá trị thứ ba còn lại

9. Nhập vào từ bàn phím thông tin về ngày X , bằng cách nhập ba số nguyên dương d , m và y lần lượt là giá trị ngày (d) tháng (m) năm (y) của X . Tính và in ra ngày tiếp sau của X

- Ví dụ: người dùng nhập $d = 31$, $m = 1$, $y = 2019$ (tức X là ngày 31 tháng 1 năm 2019) thì chương trình cần in ra 1/2/2019